



## **Assignment 1: Analysis of Sorting Algorithms**

### **SECJ2013 - DATA STRUCTURE AND ALGORITHM SEMESTER I, SESSION 2024/2025 SECTION 02**

**Lecturer: Dr. Zuraini Binti Ali Shah**

| <b>Name</b>                              | <b>Matric Number</b> |
|------------------------------------------|----------------------|
| <b>LEE YIN SHEN</b>                      | <b>A23CS0236</b>     |
| <b>AHMAD ZIYAAD BIN MOHD ABBAS</b>       | <b>A23CS0206</b>     |
| <b>ELIJAH SHE YU SHENG</b>               | <b>A23CS0073</b>     |
| <b>NURUL ASYIKIN BINTI KHAIRUL ANUAR</b> | <b>A23CS0162</b>     |

## 1.0 Analysis Report (Group Report):

### 1.1 Compare the performance (number of comparisons and swaps) of Bubble Sort, Improved Bubble Sort, Selection Sort and Insertion Sort

The performance of Bubble Sort, Improved Bubble Sort, Selection Sort and Insertion Sort was compared by the number of comparisons and swaps necessary to sort the array {75, 95, 60, 88, 70}. Bubble Sort and Improved Bubble Sort each required 10 comparisons and 6 swaps. The Improved Bubble Sort adds an optimization that stops additional iterations if no swaps take place during the process, means that the array is already sorted. In contrast, the Bubble Sort compares and swaps with neighbouring numbers. Selection Sort also required 10 comparisons but only 2 swaps since it find the largest unsorted number and switch it directly to its correct position. Insertion Sort did the fewest number of comparisons which is 9, but required 6 swaps. This is because it incrementally constructs the sorted array by inserting each element into the appropriate position within the sorted part.

### 1.2 Discuss the efficiency of each algorithm

Bubble Sort and Improved Bubble Sort is the basic algorithm that only suitable for simple dataset. But when dataset size increase, their excessive number of swaps make them become inefficient. Selection Sort is effective in reducing the number of swaps because it places the largest unsorted number directly at the correct position during each loop. However, it requires a large number of comparisons which make it less suitable for extremely large datasets. Next, Insertion Sort sorts an array of numbers by comparing and inserting number to the right place to make it simple to implement. It is efficient for nearly sorted datasets because the number of operations is reduced as fewer changes are necessary. In general, Selection Sort and Insertion Sort are preferable options for practical uses with each excelling in different scenarios depending on the input data's characteristics.

### 1.3 Table of comparison

| Sorting Algorithm    | Comparisons | Swaps |
|----------------------|-------------|-------|
| Bubble Sort          | 10          | 6     |
| Improved Bubble Sort | 10          | 6     |
| Selection Sort       | 10          | 2     |
| Insertion Sort       | 9           | 6     |